Language

Урок 21. Hilt. Многомодульность

06 октября 2022

В этом уроке обсудим один важный момент при использовании Hilt в мультимодульности

Про мультимодульность в даггере есть отдельная серия уроков. Там я по шагам создаю мультимодульный проект с даггером. Чтобы лучше понимать, что именно делает даггер под капотом, я использовал свою реализацию компонента. Также я рассматриваю различные варианты конфигурации и местоположения компонента. А здесь давайте поговорим про мультимодульность в Hilt.

При работе с Hilt нам не надо думать, какие компоненты создавать и в каких модулях их располагать. Мы только вешаем аннотации и все работает из коробки даже в многомодульном проекте. Но есть один интересный момент, который стоит обсудить.

В качестве простого примера давайте рассмотрим проект с такой структурой:



Есть три модуля app, client и data. Стрелки показывают зависимости: app знает про client, client знает про data.

MainActivity отображает ClientsFragment, который использует Database, чтобы получить данные. В проекте используется Hilt. В App класс и MainActivity добавлены необходимые Hilt аннотации.

Фрагмент тоже использует аннотации, чтобы получить Database

```
@AndroidEntryPoint
class ClientsFragment : Fragment() {

    @Inject
    lateinit var database: Database

    ...

}
```

Чтобы Hilt знал, где брать Database, мы создали Hilt-модуль:

```
@Module
@InstallIn(FragmentComponent::class)
class DataModule {

    @Provides
    fun providesDatabase(): Database {
        return Database()
    }

}
```

В нем мы поручаем фрагмент-компоненту создание Database.

На первый взгляд все в порядке. Фрагмент-компонент создаст Database и заинджектит его во фрагмент. Но есть нюанс. Давайте выясним в каком модуле находится реализация этого фрагмент-компонента.

В первых уроках про Hilt я рассказывал, что компоненты в Hilt представляют собой иерархию.
Для App создается компонент.
Для всех Activity - сабкомпонент, родителем которого является App компонент.
Для всех фрагментов - сабкомпонент, родителем которого является Activity сабкомпонент.
И так далее.

Когда даггер создает компоненты и сабкомпоненты, он делает это там, где находится основной компонент родитель всей этой иерархии. Т.е. в модуле app. Это значит, что реализация сабкомпонента для фрагмента находится в модуле app. И эта реализация создает объект Database, который находится в модуле data. Но модуль app не знает про модуль data. Если мы в любом классе в модуле app попробуем написать Database, студия не поймет, о чем мы. Классы модуля app не видят классы модуля data. Мы используем implementation зависимость, а не api.

Получается, наш сабкомпонент для фрагментов в модуле app никак не может создавать Database из модуля data, просто потому, что он ничего про него не знает.

Что-то тут явно не так. Может мы ошибаемся насчет фрагмент-компонента. Может он все таки создается где-то в недрах client модуля? (Спойлер - нет, не ошибаемся. Да и не может располагаться в модуле client компонент, который является общим для всех фрагментов в проекте)

Чтобы наверняка убедиться, что тут творится магия, давайте попробуем перенести создание объекта Database из фрагмент-компонента в App-компонент. Уж он то точно находится в модуле app.

Меняем FragmentComponent на SingletonComponent в InstallIn:

```
@Module
@InstallIn(SingletonComponent::class)
class DataModule {

    @Provides
    fun providesDatabase(): Database {
        return Database()
    }

}
```

Запускаем и все работает. App-компонент в модуле app каким то образом умудряется создавать объект Database, про который он не знает.

Разгадка находится в Hilt Gradle плагине и его механизме classpath aggregation. Этот механизм позволяет получать информацию о классах из транзитивных зависимостей, даже если мы используем implementation, а не api. В итоге сгенерированные Hilt компоненты работают с теми классами, которые не видны из модуля app.

И это никак не влияет на наш код. Мы продолжаем работать с implementation зависимостями между модулями и управлять видимостью классов так, как это нужно нам.

Здесь можно почитать подробнее про этот механизм.

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Курс Jetpack Compose



Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Назад

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.